1. What am I making?

RADIX ProofPath helps a candidate understand job readiness. Five teammates own five parts: JD Analytics -> Resume Parsing -> Profile Builder -> Talent Check -> Skill Matching. You own part 5.

Explain one job match and explore a reversible next step.

Compare one saved candidate with one saved job description. Explain matched claims, evidence gaps and the reason for each score contribution. Let the candidate try temporary improvements.

Why your part matters

A percentage alone does not tell a student what to do. The gap list and source reasoning show where the evidence does or does not cover this particular job.

What the user gives you

A saved profile and reviewed JD from Modules 3 and 1. Company data is optional and is only needed for company-readiness previews.

What you return

A match percentage, matched/missing skills and contribution details. Career GPS returns separate hypothetical results and actions without changing the saved profile.

A small example

The tiny teaching JD requires Python and the candidate has no entered skills. Match is 0%. Add hypothetical Python: projected match is 100%, while the saved profile stays empty. This describes entered evidence coverage, not true ability.

Start with the working code

This is a working starting point, not an instruction to rebuild an app. Clone the WHOLE repository so shared support is available. Work mainly in skill_matching/. Read this one guide, run it, then improve a problem you can demonstrate.

Your folder has this guide as PDF and Markdown, backend/, frontend/, tests/, samples/ and run.ps1. Shared support is stored once at the root. The Markdown contains the same teaching content and full examples as this PDF.

2. What exactly must I build?

Organizer requirements - keep these working

- Compare a profile or resume-derived skill list with one extracted JD, not a company's general reputation. This UI uses the saved profile; import a reviewed resume through Module 3 first.
- Return a 0-100 match score plus matched and missing skills. Explain gaps rather than giving only a number.
- Handle equivalent names using exact/alias/fuzzy review logic. The brief allows simple matching here; an LLM is not mandatory for this module.
- Run independently with saved input fixtures and visibly differ across candidates and JDs. Preserve the shared handoff format and the full integrated journey.

Already implemented

Alias normalization, required/preferred weights, AND/OR groups, duplicate handling, uncertain states, source decisions, GPS recalculation, ranked suggestions, reset and stale-version checks already work.

Your development checklist

- 1. Run the tiny 0-to-100 terminal example, then the richer Mira browser demo (75 to 87.5 with Docker). These are different inputs, so their scores differ.
- 2. Follow a matched and missing decision from JD words to candidate claim or absence, contribution and suggested action.
- 3. Test aliases, OR alternatives, empty JD, conflicting claims and stale revisions. Improve only a reproduced issue and add a test.
- 4. Verify GPS arithmetic, reset and identical saved profile JSON before/after. Never save a hypothetical skill or silently mark a fuzzy suggestion as matched.

Currently missing or unverified

Official sample acceptance is pending. Fuzzy suggestions need human confirmation; unknown terms can leave results provisional. GPS does not promise a learning time or a guaranteed job outcome.

Extra ProofPath features

ProofPath connects JD quote -> CV claim/absence -> score contribution -> gap -> action. GPS separately changes hypothetical named claims or category levels. Evidence confidence cannot stand in for actual competency.

3. How does it work?

1. Load pair	**2. Compare names**	**3. Score groups**
4. Explain	**5. Try a change**	**6. Reset**

- 1. Load pair: Read the current saved profile and reviewed JD. A specific job is necessary; a company name alone is not enough.
- 2. Compare names: Normalize aliases such as JS/JavaScript. Unconfirmed fuzzy suggestions or conflicting claims remain review items.
- 3. Score groups: Mandatory groups weigh 2, preferred/unspecified weigh 1. AND gets the fraction matched; OR is covered if any member matches.
- 4. Explain: Score = 100 x weighted coverage / total weight. Show the exact source, contribution and evidence gap. No JD requirements means unscorable/null, not 100%.
- 5. Try a change: GPS copies the input, adds hypothetical claims or levels and runs the same matching/readiness functions again.
- 6. Reset: Clear hypothetical changes and return to the baseline. Check the actual saved profile is identical. Stale versions require reloading the baseline.

Words you will see

Backend = Python server code. Frontend = React browser screen. API = an address the screen calls to send/receive data. A contract/schema is the agreed field list, defined in shared/contracts/models.py. An LLM is a language model that interprets text; Groq runs it. Supabase is the cloud database service.

The shared RADIX categories

COD Coding; DSA Data Structures & Algorithms; OOD Object-Oriented Design; APTI Aptitude; COMM Communication; AI Artificial Intelligence; CLOUD Cloud; SQL SQL; SWE Software Engineering; SYSD System Design; NETW Networking; OS Operating Systems.

Each skill has exactly one category, or OTHER. Descriptive skill families are extra UI filters, not new rated categories. Unknown competency is null, never zero. Confidence describes extraction support, not ability. What-if changes never save a real profile.

4. Inputs and outputs

Module 1 supplies the reviewed JD; Module 3 supplies the saved profile. The user receives matching, explanations and optional GPS previews.

POST /api/matching/evaluate

Send this complete JSON body using current saved IDs. The teaching sample has one reviewed mandatory Python requirement and an empty candidate skill list. Its complete 0% response follows; this is not the richer Mira browser fixture.

Complete input example: skill_matching/samples/input.json

```
{
  "profile_id": "59198fb6-bb16-45f0-8de6-dc3fa825fc8e",
  "jd_id": "31649844-51e6-49ba-bab7-8a2fd58d8c4d"
}
```

Important fields

- profile_id / jd_id: Both saved IDs are needed for matching. Numerical competency ratings and company benchmarks are not required for named-JD coverage.
- match_score / status: Coverage of extracted groups, not a hiring probability. complete, provisional or unscorable explains whether unresolved input remains.
- matched_skills / missing_skills: Decisions keep requirement importance, source evidence, reason and points. Missing evidence is not proof of missing ability.
- contributions / revisions: One contribution per requirement group; weights and coverage explain the total. Revisions bind calculations to the saved inputs.

POST /api/matching/scenarios/preview takes profile_id, jd_id, profile_revision, jd_revision, optional skill_overrides/category_overrides and optional company selection. samples/scenario-request.json is a full request; scenario-expected.json is a labelled result summary, not the complete ScenarioResult. The full result contains baseline/projected match, optional baseline/projected readiness, deltas, recommendations and a notice. Preview never updates Profile.

The next page(s) show the COMPLETE captured response from the existing API, including defaults and empty fields. Long lines wrap for reading. Use samples/output.json for exact machine-readable JSON. IDs, receipt IDs, source hashes and times can differ between runs; do not hardcode them. JSON null means unknown/not present; [] means an empty list.

Complete output example

skill_matching/samples/output.json


```
{
  "jd_source_file": "practice-jd-SYNTHETIC.txt",
  "jd_id": "31649844-51e6-49ba-bab7-8a2fd58d8c4d",
  "jd_revision": 1,
  "profile_id": "59198fb6-bb16-45f0-8de6-dc3fa825fc8e",
  "profile_revision": 1,
  "match_score": 0.0,
  "matched_skills": [],
  "missing_skills": [
    {
      "requirement_id": "88b11eeb-6843-40f9-9958-4c3ab27a0af0",
      "skill_name": "Python",
      "category_code": "COD",
      "matched": false,
      "requirement_type": "mandatory",
      "jd_evidence": {"skill_id": "790da7da-908f-4536-a1ba-7ddceedd0ac3", "skill_name":
  "Python", "category_code": "COD", "skill_family": "Programming Languages", "evidence":
  "Required: Python", "confidence": "medium", "source_page": null, "source_file":
  "practice-jd-SYNTHETIC.txt", "source_document_id": "f12bd631-d25d-47aa-
  ab45-ac34c3bd8281", "source_type": "document", "evidence_type": "jd_requirement",
  "verification_status": "unverified", "claim_status": "claimed", "claim_basis":
  "explicit", "normalized_name": null, "source_start": null, "source_end": null},
      "candidate_evidence": null,
      "decision_type": "missing",
      "reason": "No equivalent reviewed skill claim found",
      "suggested_review": [],
      "recommended_action": "Review source coverage for Python; demonstrate existing
  ability or develop it if needed",
      "score_contribution": 0.0
    }
  ],
  "ambiguous_matches": [],
  "contributions": [
    {"requirement_id": "88b11eeb-6843-40f9-9958-4c3ab27a0af0", "operator": "AND",
  "requirement_type": "mandatory", "skills": ["Python"], "weight": 2, "coverage": 0.0,
  "weighted_contribution": 0.0, "requires_review": false, "member_count": 1,
  "matched_count": 0, "score_contribution": 0.0}
  ],
  "rules_version": "match-1.3-incomplete-extraction-disclosure",
  "status": "complete",
  "methodology": "Weighted extracted requirement-group coverage: mandatory 2,
  preferred/unspecified 1 (team heuristic). AND fraction, OR any. Unparsed requirement
  warnings keep the result provisional until reviewed. Confidence is not competency;
  named technologies require equivalent claims.",
  "receipt_id": "22b84051-f5d3-4e82-b1ae-0f7733770fe6"
}
```

5. How do I work on the code?

Install and run - Windows PowerShell

Install Python 3.10+, Node.js 24 LTS (includes npm), Git and an editor such as VS Code. Accept the installers' PATH option, reopen PowerShell and clone the full GitHub repository. Open a terminal at the repository root - the folder containing package.json.

```
.\skill_matching\run.ps1 setup
# Terminal 1:
.\skill_matching\run.ps1 api
# A SECOND terminal, same root:
.\skill_matching\run.ps1 web
```

Open http://127.0.0.1:5173/. Server API help is at http://127.0.0.1:8000/docs. setup creates .venv and installs locked Python/npm dependencies plus Chromium for browser tests. Run setup once; no key is needed. Keep api and web running; Ctrl+C stops your own server. Only one module uses 8000/5173 at a time.

Open these files first

- skill_matching/backend/matcher.py - Open first. Name aliases, requirement groups, contributions and matching decisions.
- skill_matching/backend/scenarios.py - Copies input, recalculates previews and ranks single-change suggestions.
- skill_matching/backend/router.py - Loads saved inputs, checks versions and exposes evaluate/preview routes.
- skill_matching/frontend/SkillMatchingPage.tsx - Shows scores, decisions, controls and reset. ReasoningChain.tsx shows source-to-action explanations.

shared/contracts/models.py defines MatchingResult and ScenarioRequest/Result. talent_check/backend/scoring.py and benchmark_repository.py are existing pure-code dependencies in the same clone; no second teammate server is needed. shared/integrations/receipts.py keeps result snapshots.

Make one useful change

Read a failing example, locate the responsible function, change that part and add a test in your tests/ or frontend test file. Do not rename shared fields, duplicate common code or rewrite other modules. Ask the Scrum Master before changing shared contracts, storage or provider code.

What should I see?

Expect a candidate/job pair, Analyze this match, matched and gap lists, a clickable explanation chain, and Career GPS with baseline/projected scores and Reset. Keep job coverage and company readiness clearly named.

Load synthetic inputs supplies pre-reviewed Mira records for independent operation. It is not fresh extraction. The tiny Asha terminal examples and the richer Mira browser examples are intentionally different inputs.

6. How do I know it works?

```
.\skill_matching\run.ps1 example
.\skill_matching\run.ps1 test
```

example runs the small teaching case in a fresh temporary database and checks its result. test runs your real standalone API check, Python tests, React tests, type checking, lint, your production build and your browser test. It stops on failure. Both use synthetic data; tests force rules mode and do not prove live LLM quality.

- Tiny pair: samples/profile.json has no entered skills; jd.json requires Python. The example command must return 0%, then 100% after hypothetical Python, with saved profile unchanged.
- Richer browser pair: Load synthetic inputs and analyze Mira/Junior Data Engineer: 75%. Hypothetical Docker gives 87.5%. Reset returns 75%. These fixtures differ from the tiny JSON example.
- Source reasoning: Open a Python match in the richer pair: its two-word mandatory AND group gives Python 12.50 points. Open Docker's gap: no equivalent claim and zero contribution.
- Aliases / alternatives: Tests compare JS/JavaScript and AWS OR GCP. An alias may match; a spelling suggestion must not silently earn points. One OR member is enough.
- Empty / stale / unchanged: Empty JD gives null/unscorable; a stale revision gives 409. Check saved profile JSON before/after scenarios and reset; any added hypothetical claim is a bug.

Use an unfamiliar document too

Use only documents you own or have permission to process. Before running extraction, list expected skills and supporting words yourself. Compare missing and invented skills, sections, groups and quotes. A successful API response or valid JSON is not proof of correct extraction. Do not commit real CVs or send them to Groq without explicit permission.

If something fails

Read the FIRST error. Missing Python/npm: install and reopen the terminal. Missing packages: rerun setup. Browser cannot reach API: keep api running. Port busy: stop your own earlier server or coordinate a free port; do not kill unknown processes. Wrong extraction: compare the source, review and add a regression case.

7. Missing resources and what to do

What you can do now

All local teaching inputs are synthetic. No organizer pack, API key or cloud account is needed to run, edit and test your module. The tiny samples teach fields and behavior; the larger reconstructed pack remains under data/synthetic/. Neither is official organizer data.

Your samples - every file has a purpose

- input.json - Complete upload description or JSON request shown in this guide.
- jd.json - Manually reviewed teaching JD requiring Python; not a model extraction claim.
- output.json - Complete response captured from that actual API request.
- profile.json - Saved synthetic beginner profile with no entered skill claims.
- scenario-expected.json - Checked summary: 0 -> 100 matching; saved profile unchanged. Not a full API response.
- scenario-request.json - Full what-if request adding hypothetical Python.

Other modules and cloud resources

The shared folder and locked root dependencies are required; clone the whole repository. The local workspace supplies reviewed upstream snapshots, so you do not wait for a teammate's server. Later, integration uses real saved IDs, versions and owned source links, not the snapshot UUIDs in this guide.

Supabase accounts, live database rules and real organizer data are configured and verified by the Scrum Master when authorized. Local SQLite storage already works. Module 3 needs no LLM; Module 5's named-JD matching also needs no LLM or company data. Company-readiness previews remain unavailable without a valid chosen benchmark.

8. Submit and explain your module

Completion checklist

[ ] I can run/explain it. [ ] Required inputs/outputs work. [ ] Normal/failure cases checked. [ ] Tests passed or failures reported. [ ] Contracts/source links preserved. [ ] No secrets/private CVs committed.

Push to YOUR own GitHub

Create an EMPTY private repository on your own account. From your full working clone, replace the URL below with YOUR new repository, review/stage only your changes, then run:

```
git remote rename origin scrum-master
git remote add origin https://github.com/YOU/YOUR-REPO.git
git add skill_matching
.\.venv\Scripts\python.exe scripts/scan_handoff.py --staged
git commit -m "Verify my RADIX module"
git push -u origin main
```

Keep the whole working clone, including your source/tests/samples, guides, launcher and shared support. Never force-push. Grant the Scrum Master access if private.

Return URL, git rev-parse HEAD, changed files, actual test commands/results, three normal + three failure cases, synthetic screenshots and limitations. Distinguish implemented, mock-tested, live-verified and pending. The Scrum Master handles integration; you need not solve central merges.

A 45-second explanation for judges

My part compares a candidate with one actual job description and explains the result. It scores reviewed requirement groups, handles aliases and alternatives, and shows matched evidence and gaps. Each decision connects the JD words to a candidate claim or absence and its score contribution. Career GPS tries temporary changes and reruns the same mathematics. Reset restores the baseline, and saved profile data never changes. Confidence is separate from ability, and the score measures extracted requirement coverage rather than a hiring guarantee.

Three likely questions

- Does 100% guarantee selection? No. It means coverage of the extracted, reviewed requirement groups for these inputs.
- Can a what-if change my real skills? No. It uses a temporary copy; saved profile equality is tested.
- Is company readiness the same score? No. Named-JD coverage and twelve-category self-rating comparison use different inputs and formulas.

Copy this prompt into your AI assistant

Help me complete module 5: Skill Matching + Career GPS step by step using this guide. First run it; request the exact section-5 source files and tests before editing. Preserve working code and shared fields. Fix a reproduced issue and test it. Keep confidence separate from competency and scenarios separate from saved profiles. Never invent official data or request pasted keys. Help me submit my own GitHub URL, tested commit, results and honest limitations.